

Vibe-Code Blueprint

An Architecture Conformance Layer for AI-Assisted Software Development

Project Proposal and Technical Reference Document
Prepared for mentor review and as the primary development reference
August 2026

Elevator Pitch

Vibe-Code Blueprint is a local-first developer tool that reads any AI-generated codebase and shows the architecture that actually exists — then measures it against the architecture the AI said it was building.

When a coding agent promises a clean layered design and then quietly violates it forty files later, the developer currently finds out weeks afterwards, during a painful refactor. Vibe-Code Blueprint surfaces that gap in seconds, ranks the violations by severity, and tracks how far the architecture has drifted over time.

The tool never generates code. It measures. That single design decision is what makes it trustworthy, technically achievable by one developer, and scientifically publishable.

The problem in one table

	TODAY (WITHOUT THE TOOL)	WITH VIBE-CODE BLUEPRINT
Visibility	Nobody knows what the architecture actually is. The only description is a stale README or a chat message from three weeks ago.	A live, derived architecture graph generated directly from the code, in under a minute.
Intent tracking	The agent said "we will use a repository pattern." Nobody ever checked whether it did.	Stated intent is extracted automatically and compared against the real code.
Drift	Erosion is discovered only when a feature becomes impossible to add.	A drift score is tracked per commit and charted over time.
Correction cost	By the time drift is visible, fixing it is a multi-week refactor across dozens of files.	Violations are caught within the same session, when they cost minutes to fix.

Scope clarity: what this is and what it is not

THE TOOL IS	THE TOOL IS NOT
A measuring instrument for software architecture	A code generator or a compiler
A verification layer that sits after any coding agent	A replacement for Cursor, Claude Code or Kiro
A local process; code never leaves the machine	A cloud service or a hosted SaaS product
An MCP context provider that any agent can query	An IDE or an editor

Section 0 — Core Strategy: Type-2 as the Foundation

0.1 Market position

The forward direction of this idea — write a structured specification first, then let an agent generate code from it — already exists as a named and crowded category called **Spec-Driven Development (SDD)**. GitHub Spec Kit was open-sourced in September 2025 and gave the category its name. By 2026, AWS Kiro, TESSL, BMAD, OpenSpec, Cursor Plan Mode, Claude Code and Google Antigravity have all shipped their own versions.

Entering that space directly would mean competing with well-funded incumbents who have a two-year head start. **We therefore do not compete on generation. We compete on verification.**

This position is supported by the incumbents' own admissions. TESSL acknowledges that the same specification produces different code from different agents. Thoughtworks' Birgitta Böckeler has warned that these tools risk creating review overload and a false sense of control over non-deterministic models. The pragmatic consensus that emerged through 2026 is **spec-anchored, not spec-as-source**: code remains the source of truth, and verification is the enforcer.

Vibe-Code Blueprint is built exactly for that consensus position. Every SDD tool on the market produces intent. None of them measure whether the intent survived contact with the generated code.

0.2 Why Type-2 (Reverse-Blueprint) is the foundation

DIMENSION	TYPE-2 (REVERSE) — BUILD FIRST	TYPE-1 (FORWARD) — BUILD LATER
User commitment needed	None. Point it at an existing repo and get value immediately. No behaviour change required.	High. The user must author a blueprint first and trust that it pays off later.
Time to first value	Under one minute	Only after a full generation cycle
Market crowding	Sparse. Reverse architecture extraction for AI-generated code is largely unserved.	Saturated. Eight or more funded competitors.
Shareability	High. "Run this on your repo and see what it finds" is a message people forward.	Low. Requires explanation before it means anything.
Research value	High. Produces measurable conformance data.	Low. Generation quality is hard to attribute.
Dependency on others	None. Works with any agent, any workflow.	Requires deep MCP integration and agent cooperation.

0.3 The unifying architectural insight

Type-1 and Type-2 are the same engine.

Both reduce to a single operation: *compare an intended architecture against an actual architecture and report the difference*. The only thing that changes is where the intent came from.

- In **Type-2**, intent is extracted from chat logs, AGENTS.md, README or ADRs.
- In **Type-1**, intent is authored directly by the user.

We build the comparison engine once. Type-1 then becomes a thin authoring layer on top of an engine that is already proven, rather than a second product.

Consequence for the roadmap: Type-1 is deliberately deferred beyond the MVP. When it is added, its final pipeline step is to run the Type-2 engine on the generated code. **The forward mode literally ends by invoking the reverse mode.**

Section 1 — Platform Strategy

1.1 The decision: Local CLI plus Browser UI

The developer runs a single command inside their repository. A local server starts and opens a browser interface. This is the same pattern used by Jupyter, Streamlit and Storybook — a proven model for developer tools that need both file system access and rich rendering.

```
$ npx vibe-blueprint .

Scanning repository ..... 1,284 files
Parsing (ts, py) ..... done
Resolving dependencies ..... 3,902 edges
Clustering modules ..... 23 modules
Extracting stated intent .... 14 constraints found
Checking conformance ..... 9 violations

-> http://localhost:5173
```

1.2 Why this beats SaaS and Desktop for a solo developer

OPTION	FILE ACCESS	GRAPH RENDERING	LLM INTEGRATION	INFRASTRUCTURE COST	VERDICT
Web SaaS	Poor. File System Access API is Chromium-only and permission-gated. Uploading a repository is unacceptable UX.	Excellent	Requires a proxy, key management and billing	Very high. Authentication, multi-tenancy, hosting, database ops, security review	Rejected
Desktop app (Tauri / Electron)	Excellent	Good	Excellent	High. Packaging, code signing, auto-update, three OS targets	Rejected
Local CLI + Browser UI	Native and unrestricted	Excellent — full browser capability	User's own API key; zero infrastructure	Effectively zero	SELECTED

The three decisive advantages

- **Privacy is a product feature, not a compromise.** "Your source code never leaves your machine" removes the single largest objection to adopting a tool that reads an entire codebase. For any future enterprise user, this is worth more than any feature on the roadmap.
- **Zero infrastructure.** No authentication, no multi-tenancy, no hosting bill, no database operations, no abuse handling, no rate-limit engineering. Realistically this removes around 40% of the total engineering work, which is precisely what makes a 3-month solo MVP achievable.
- **Superior UX for the actual use case.** The developer is already in the terminal, inside the repository. One command is less friction than opening a website, authenticating and uploading a project.

Acknowledged trade-off: no real-time collaboration, harder monetisation, and no hosted demo. This is mitigated by a **static HTML export** — the blueprint renders to a single self-contained file that can be shared or committed to the repository. This provides the shareable artefact without building any hosting.

1.3 Phase 2: The VS Code Extension

Once the local engine is proven, the natural next surface is a **VS Code extension** built on the same core. This is an extension of the strategy, not a replacement for it.

ASPECT	DETAIL
Architecture	The extension does not reimplement anything. It spawns the existing CLI as a local language-server-style process and communicates over JSON-RPC. One engine, two front-ends.
What it adds	Inline violation warnings in the editor gutter · a sidebar architecture tree · "why is this flagged?" on hover · a live drift indicator in the status bar · one-click jump from a violation to the offending import statement.
Distribution advantage	The VS Code Marketplace provides discovery that a CLI cannot. Install count also becomes a public, verifiable adoption metric — useful for both the research write-up and the CV.

ASPECT	DETAIL
Strategic value	It places the tool at the exact moment the violation is introduced, rather than after the fact. Detection moves from retrospective to real-time.
Why not first	Webview rendering constraints make heavy graph work harder, and the extension API adds learning overhead. Building the engine standalone first keeps it front-end agnostic.

1.4 Deployment progression

STAGE	SURFACE	TIMING	PURPOSE
Stage 1	Local CLI + Browser UI	MVP (Weeks 1–12)	Prove the engine. Zero infrastructure.
Stage 2	VS Code Extension	Month 4–5	Distribution and real-time detection.
Stage 3	CI/CD integration (GitHub Action)	Month 5–6	Conformance as a merge gate.
Stage 4	Optional team cloud sync	Only on user demand	Shared architecture history.

Section 2 — Product Features

2.1 Core capabilities (MVP scope)

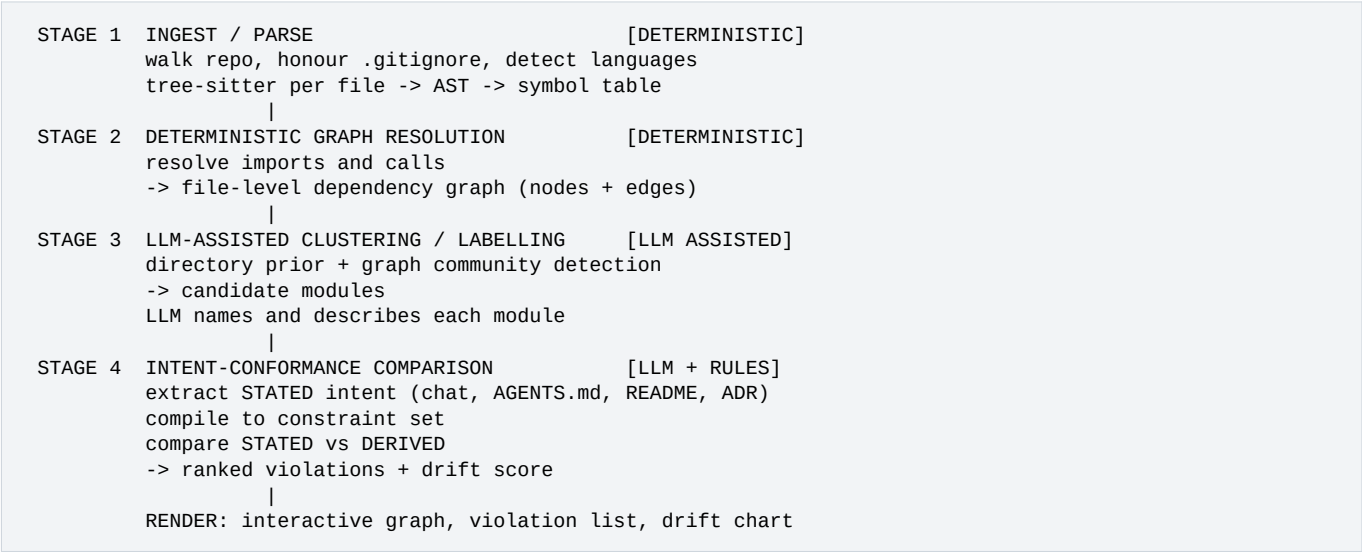
CAPABILITY	DESCRIPTION	VALUE TO USER
Architecture extraction	Derives a live module-level architecture graph directly from source code using deterministic static analysis.	See what actually exists, not what the README claims.
Intent extraction	Automatically reads stated architectural intent from chat logs, AGENTS.md, README files, ADRs and commit messages.	The promises become machine-checkable.
Conformance checking	Compares stated intent against derived reality and produces a ranked violation list.	"Here are 9 places where the code contradicts what was promised."
Semantic diffing	Diffes architecture at node and edge level rather than text level. Reports structural change in plain language.	"Module auth gained a dependency on billing."
Drift scoring	A single quantitative score per commit, charted over time.	Erosion becomes visible before it becomes fatal.
MCP server	Exposes the architecture as queryable context over Model Context Protocol.	Any agent can ask "am I allowed to import this?" <i>before</i> writing code.
Static HTML export	Renders the full interactive blueprint into one self-contained file.	Shareable and committable, with no hosting required.

2.2 Post-MVP feature roadmap

FEATURE	WHAT IT DOES	WHY IT MATTERS
Constraint-based editing	The user states executable rules ("domain must not import from infra") instead of drawing boxes. The graph is derived from the rules.	A drawn diagram is a picture and goes stale. A constraint is executable. This is what ArchUnit, Deptrac and go-arch-lint already do textually; we make it visual and LLM-authorable.
Architectural blame	Identifies which commit — and which prompt — introduced a given violation.	Uniquely possible when chat logs are available. No existing tool can do this.
Conformance CI gate	Architecture violations fail the pull request.	Converts documentation into enforcement.
Drift-to-refactor plan	Generates an ordered task list that fixes detected drift, as MCP-consumable tasks.	Closes the loop from detection to correction. This is where willingness-to-pay lives.
Multi-agent divergence report	Gives the same blueprint to several agents and measures how differently each implements it.	Directly productises the inconsistency TESSl publicly admits. Nobody is measuring this today.
Time-travel replay	A slider that animates architectural evolution across commit history.	Excellent demonstration value and genuinely useful for onboarding new developers.
Sketch-to-blueprint	Converts a whiteboard photograph into a structured blueprint draft using a vision model.	Captures design discussions at the moment they happen.

Section 3 — System Architecture

3.1 The four-stage pipeline



The determinism boundary is the most important rule in this system.

Stages 1 and 2 are **fully deterministic**. The language model never invents structure. It only *names* clusters and *reads intent from prose*.

If a model were allowed to guess the dependency graph, a single hallucinated edge would destroy user trust permanently. Every architectural fact we display must be traceable to a real import statement in a real file.

3.2 The central data structure: the Architecture Graph

Node	: id, kind (module service layer), label, files[], provenance
Edge	: from, to, kind (imports calls depends), evidence[], count
Constraint	: subject, relation (must-not-import may-only-via), object, source, confidence
Violation	: constraint_ref, edge_ref, severity, first_seen_commit
Provenance	: every fact tagged DERIVED (from code) or STATED (from intent)

The provenance field is a deliberate design decision, not a detail. It keeps facts derived from code strictly separated from claims made in prose. Without it, model guesses contaminate ground truth and the entire tool becomes unreliable. Every element rendered in the UI is visually marked according to its provenance.

3.3 Technical stack

LAYER	TECHNOLOGY	REASON
CLI / runtime	Node.js (TypeScript)	Single distribution via npx; same language as the UI; strong tree-sitter bindings.
Parsing	tree-sitter (TypeScript and Python grammars)	Fast, incremental, error-tolerant. Handles broken and half-finished code, which is normal in vibe-coded repos.
Graph engine	graphology + Louvain community detection	Mature clustering with good performance at repository scale.
Storage	SQLite (local file)	Zero setup, versioned snapshots, no server process.
UI	React + React Flow (Sigma.js fallback)	Best-in-class node graph rendering. WebGL fallback for very large graphs.
LLM access	User-supplied API key; provider-agnostic adapter	No proxy, no billing, no vendor lock-in.

LAYER	TECHNOLOGY	REASON
Agent integration	MCP server + AGENTS.md file export	Dual transport protects against protocol churn.

Section 4 — Feasibility and Bottlenecks

4.1 The primary risk: round-trip synchronisation

The problem. A system that keeps a visual blueprint and a codebase in two-way agreement is attempting round-trip engineering. The industry pursued exactly this through UML and Model-Driven Architecture for roughly thirty years and largely failed. Two sources of truth that both accept edits will always diverge, and reconciling them is an unbounded problem.

Our mitigation: we do not attempt to solve it. We design it out of the product.

Vibe-Code Blueprint is a measuring instrument, not a compiler.

The code remains the single source of truth at all times. The blueprint is a *checked claim* about the code, never an authoritative twin of it. We never generate code from the blueprint. We hand the blueprint to the user's own agent as context, and then we measure the result.

This is a smaller promise than round-trip synchronisation — and a far more defensible one. It also aligns precisely with the market consensus of spec-anchored rather than spec-as-source.

4.2 Secondary bottlenecks and mitigations

BOTTLENECK	THE PROBLEM	MITIGATION
Semantic grouping of messy repos	Syntax parsing is solved. Deciding which 40 files constitute "the auth module" is not. Vibe-coded repositories have no consistent structure by definition.	Hybrid approach in order: directory structure as prior, then import-graph community detection, then LLM labelling, then a user correction UI . Target "good enough that the user wants to argue with it" — a wrong-but-close diagram that takes two minutes to fix beats a blank canvas. Corrections persist.
Context window limits	A repository with 100,000 files cannot be placed into a prompt.	Never send code to the model — send the graph. That same repository has perhaps 200 modules. Sending 200 labelled nodes costs a few thousand tokens. Code snippets go only for the single cluster being labelled. This makes the system scale-independent.
Agents ignore the blueprint	Even with perfect context, non-deterministic generation deviates. An agent can produce 50 files that each look correct individually but collectively break the design.	Sell detection, not control. We never claim the agent will comply — that claim would be tested and fail immediately. We claim the developer will know within seconds when it did not.
Graph rendering performance	React Flow degrades badly beyond roughly 1,000 nodes.	Hierarchical collapse — render one level at a time and expand on demand. Users cannot comprehend 1,000 nodes anyway, so this is better UX regardless. Sigma.js WebGL fallback for full-graph views.
MCP protocol churn	The protocol is still evolving and may change.	Abstract the transport layer. Support MCP and plain-file export. A protocol change means rewriting one adapter, not the product.
Language sprawl	Supporting many languages is the classic solo-developer trap.	TypeScript/JavaScript and Python only for the MVP. This covers the large majority of vibe-coded projects. Additional languages only on demonstrated user demand.

Section 5 — Research Contribution and CV Impact

5.1 The thesis

The Agentic Conformance Thesis

"Architectural Conformance in LLM-Generated Code: Do Coding Agents Build What They Say They Will Build?"

The field currently measures what models **say**. Almost nobody measures what agents **do**. This project measures the gap.

5.2 Why this is genuinely novel

Architecture conformance checking is a mature academic field. Reflexion Models, Source Code Query Languages and Dependency Structure Matrices are the three established static approaches. Software Architecture Reconstruction has extracted architecture from code for two decades specifically to manage drift.

However, every one of these methods assumes the intended architecture was authored by a human architect. That assumption held for thirty years and broke in 2024.

Today the intended architecture is frequently **stated by a language model in a chat conversation** — and then violated by that same model twenty messages later. To our knowledge, no published work has:

- Treated the **chat transcript as a formal architectural intent artefact**
- Measured **conformance between what an agent stated it would build and what it actually built**
- Quantified how that gap **grows over the course of a single session**

The artefact required for this research — a conversational log paired with the resulting codebase — simply did not exist before 2024. That is what makes the contribution new rather than incremental.

5.3 Study design

STEP	METHOD
1. Corpus	Collect 50–100 public repositories that contain chat logs, AGENTS.md files or documented agent sessions alongside the code.
2. Intent extraction	Automatically extract stated architectural constraints from the conversational and documentation artefacts.
3. Reality derivation	Derive the actual architecture graph using deterministic static analysis.
4. Conformance measurement	Compute violation rate per constraint type, per agent, per project age.
5. Analysis	Report drift growth against session length, differences across agents, and which categories of intent are most frequently abandoned.

Target headline result: *"Agents stated 340 architectural constraints across 80 projects and violated 61% of them, with violation rate rising sharply after message 30."*

5.4 Expected contributions and CV impact

CONTRIBUTION	TYPE	VALUE
First conformance benchmark for agent-generated architecture	Dataset	Reusable by others, which attracts citations
Empirical drift measurement across coding agents	Empirical result	Directly useful to every team deploying agents
Open-source verification tool	Artefact	Demonstrates engineering ability, not only analysis
Extension of Reflexion Models to machine-authored intent	Theoretical	Connects new work to established literature

Why this combination is strong on a CV: developer tooling paired with empirical measurement is rare. Most candidates produce either a tool with no evaluation or an analysis with no artefact. This project produces both, and the topic is directly relevant to nearly every company now deploying coding agents.

Risk containment: a product can fail to find users. A measurement study cannot fail to produce a result — even a finding of "agents comply more than expected" is publishable and interesting. Sequencing the work so that the research data is a by-product of building the tool caps the downside of the entire project.

Section 6 — Three-Month MVP Roadmap

6.1 Phase overview

	PHASE 1 (WEEKS 1–4)	PHASE 2 (WEEKS 5–8)	PHASE 3 (WEEKS 9–12)
Theme	CLI and Parser	Conformance Engine	Drift Analysis and Publication
Goal	See the actual architecture	Compare against stated intent	Track change over time
Demo statement	"Here is what your repository actually looks like."	"Here are 9 places where the code contradicts what was promised."	"Here is how your architecture eroded over four months."
If running late	Cannot cut. Everything depends on this.	Cut intent-from-chat-logs; keep AGENTS.md and README only.	Cut the MCP server; keep static HTML export.

6.2 Week-by-week schedule

WEEK	FOCUS	DELIVERABLE
1	Project setup, CLI scaffold, repository walker	<code>npx vibe-blueprint</code> . runs and lists files, honouring <code>.gitignore</code>
2	tree-sitter integration for TypeScript and JavaScript	AST and symbol table produced for every TS/JS file
3	Python grammar; import and call resolution	File-level dependency graph for both languages
4	Basic React Flow rendering; local server	Milestone 1: browser shows a real dependency graph of any repository
5	Module clustering (directory prior + Louvain)	Candidate modules identified and grouped
6	LLM labelling; user correction UI	Modules carry meaningful names; corrections persist to SQLite
7	Intent extraction from AGENTS.md, README and ADRs	Stated constraints parsed into a structured constraint set
8	Constraint compiler and violation detection	Milestone 2: ranked violation list with evidence links
9	Commit-snapshot versioning; semantic diff	Architecture compared between any two commits
10	Drift scoring and timeline chart	Drift score computed per commit and plotted over history
11	MCP server; static HTML export; corpus collection begins	Milestone 3: agents can query architecture; blueprint is shareable
12	Run study across corpus; write paper; documentation and release	Final: public repository, README, demo, and paper draft

Weeks 11 and 12 run the research and the product in parallel. The drift data generated to validate the tool *is* the research dataset. This is the point where the two goals stop competing for time and begin reinforcing each other.

6.3 Explicitly out of scope for the MVP

Visual drag-and-drop editing · voice input · sketch-to-blueprint · Type-1 blueprint authoring · collaboration features · authentication · cloud synchronisation · languages beyond TypeScript and Python · blueprint-by-example · VS Code extension.

Every item on this list is a good idea, and several are on the post-MVP roadmap. Attempting any of them before Week 12 is the most likely cause of the project failing to ship.

Why This Is a Category Leader for 2026

The 2026 AI developer tool market is crowded on the generation side and nearly empty on the verification side. Five structural reasons make this position defensible.

#	ARGUMENT	EXPLANATION
1	We sit on the empty side of a crowded market	Every major player — Spec Kit, Kiro, TESSL, Cursor, Antigravity — competes on producing intent. None measure whether the intent survived. We are not a smaller version of a competitor; we are the complementary half the category is missing.
2	The problem grows automatically	The volume of agent-generated code is rising steeply, and architectural drift compounds with volume. Our addressable problem expands without any effort on our part. Very few product positions have this property.
3	Local-first is a durable moat, not a limitation	As agent adoption reaches regulated and enterprise environments, "your code never leaves your machine" shifts from a nice feature to a procurement requirement. Competitors built on cloud architectures cannot retrofit this.
4	We become infrastructure rather than another IDE	By exposing architecture over MCP, we are consumed by other tools instead of competing with them. Claude Code, Cursor and Kiro become distribution channels rather than rivals. This is a far stronger strategic position than building a competing editor.
5	The data asset compounds	Every conformance run adds to a growing corpus of agent behaviour data. That corpus supports the research contribution, improves the product's heuristics, and becomes progressively harder for a new entrant to replicate.

Summary

Vibe-Code Blueprint occupies a specific and defensible position: it is the **verification layer for the spec-driven development era**. It requires no behaviour change from the developer, works with any coding agent, keeps all code local, and produces a measurable research contribution as a by-product of its normal operation.

It is scoped so that a single developer can deliver a working, demonstrable MVP in twelve weeks — and scoped so that even a commercial failure still yields a publishable scientific result.

Open Questions for Mentor Discussion

- **Corpus availability:** how many public repositories genuinely contain usable agent chat logs? If the number is low, we may need to generate the corpus ourselves by running controlled agent sessions.
- **Constraint extraction accuracy:** what is an acceptable precision threshold for automatically extracted intent? A false violation is more damaging to user trust than a missed one.
- **Publication venue:** is this a better fit for a software engineering venue (ICSE, FSE, MSR) or an AI-for-code workshop? MSR appears well matched, given the mining-repositories methodology.
- **Scope confirmation:** is deferring Type-1 entirely beyond the MVP acceptable from an evaluation standpoint, given that the original concept described both directions?